Análisis visual de datos

M4: Análisis exploratorio de datos

|**AE5**: Aplicar técnicas de análisis visual de los datos utilizando la librería seaborn para la representación gráfica de la información.

alkemy

Introducción👩‍💻

En la era del big data, el análisis visual de los datos se ha convertido en una herramienta esencial para la comprensión, exploración y comunicación de información. Una correcta representación gráfica no solo permite identificar patrones, relaciones o anomalías en los datos, sino que también facilita la toma de decisiones fundamentadas. En el ecosistema de Python, la librería Seaborn se destaca por su capacidad para generar gráficos estadísticos de alta calidad con un enfoque intuitivo y atractivo visualmente.

Este manual tiene como objetivo brindar una guía exhaustiva sobre el uso de Seaborn para el análisis visual de datos. Abordaremos desde sus características principales hasta la construcción de distintos tipos de gráficos, explicando paso a paso cada funcionalidad con fundamentos teóricos, diferencias con otras técnicas, y recomendaciones de uso según el tipo de dataset.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Aplicar técnicas de análisis visual de los datos utilizando la librería Seaborn.
- Identificar las principales características de la librería Seaborn para el análisis visual.
- Importar y preparar Seaborn para trabajar con datasets estadísticos.
- Seleccionar el tipo de gráfico que mejor se adapte a las necesidades de información.
- Construir distintos tipos de gráficos con Seaborn, personalizando su presentación.
- Interpretar visualmente relaciones, distribuciones y comparaciones entre variables numéricas y categóricas.
- Representar matrices de correlación y distribución con mapas de calor.
- Utilizar grillas de gráficos para representar múltiples variables de forma conjunta.

Características de la librería Seaborn

Seaborn es una librería de visualización estadística construida sobre Matplotlib, que proporciona una interfaz de alto nivel para crear gráficos informativos y atractivos. Sus principales características son:

Característica	Descripción
Integración con pandas	Trabaja perfectamente con DataFrames y Series.
Estética mejorada	Gráficos más profesionales y pulidos por defecto.
Gráficos estadísticos	Incorpora funcionalidades estadísticas como líneas de regresión, intervalos de confianza, distribución de frecuencias, etc.
Paletas de colores integradas	Permite aplicar fácilmente paletas cromáticas temáticas.
Soporte para múltiples ejes	Con PairGrid y FacetGrid se pueden crear visualizaciones complejas en grillas.

A diferencia de Matplotlib, que requiere mayor configuración manual para personalizar los gráficos, Seaborn ofrece funciones preconfiguradas que permiten obtener resultados impactantes con pocas líneas de código.

Importación de la librería

Para comenzar a utilizar Seaborn, es necesario importarla junto con sus dependencias más comunes:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

En la mayoría de los casos, Seaborn ya está disponible si se trabaja en entornos como Jupyter Notebook o Google Colab. De no estar instalada, se puede instalar mediante:

```
pip install seaborn
```

También es habitual configurar el estilo general de los gráficos desde el inicio:

```
sns.set(style="darkgrid")
```

Esto establece un fondo gris con líneas de grilla, muy útil para visualizar tendencias.

Tipos de gráfico

Seaborn permite generar una amplia variedad de gráficos, agrupados según la naturaleza del análisis:

Tipo de gráfico	Funciones asociadas	Uso principal
Distribución univariada	distplot, histplot, kdeplot	Distribución de una variable numérica
Relación bivariada	scatterplot, regplot, jointplot	Relación entre dos variables
Comparación categórica	barplot, countplot, boxplot, violinplot	Comparación entre grupos categóricos
Correlación/matriz	heatmap, clustermap	Visualización de matrices y relaciones

Múltiples gráficos	pairplot, PairGrid, FacetGrid	Visualización de múltiples variables

Cada función tiene sus parámetros propios, pero también comparte configuraciones generales como hue, palette, style, size, lo cual permite personalizar completamente las visualizaciones.

Gráficos de distribución de observaciones: distplot

Nota importante: distplot está obsoleta desde Seaborn v0.11. Se recomienda utilizar displot() o histplot().

distplot era la función tradicional de Seaborn para graficar histogramas de frecuencia con una curva de densidad ajustada. Se usaba así:

```
sns.distplot(df['edad'])
plt.show()
```

Hoy en día, se recomienda:

```
sns.histplot(df['edad'], kde=True)
plt.show()
```

Parámetros importantes:

- kde=True: activa la curva de densidad.
- bins=30: cantidad de intervalos.
- color='skyblue': color del histograma.

Comparación:

Función	**Vigente**	**Curva KDE**	**Personalización fácil**
distplot	No	Sí	Media

histplot	Sí	Sí	Alta
displot	Sí	Sí	Muy alta (grid opcional)

Conclusión: para análisis actuales, es recomendable usar histplot para gráficos univariantes o displot para múltiples subgráficos.

Gráficos de dispersión y correlación de variables: jointplot

jointplot() combina un gráfico de dispersión con histogramas marginales, mostrando cómo se relacionan dos variables numéricas y su distribución individual.

```
sns.jointplot(data=df, x='altura', y='peso', kind='scatter')
```

Tipos (kind=):

Tipo	Descripción
scatter	Dispersión tradicional
kde	Densidad bivariada
hex	Hexbin (agrupamiento por densidad)
reg	Dispersión con línea de regresión

Ventajas:

- Muestra relaciones y distribuciones al mismo tiempo.
- Fácil de interpretar para detectar patrones.

Ejemplo con regresión:

```
sns.jointplot(data=df, x='ingresos', y='gastos', kind='reg')
```

Ideal para estudios económicos, salud o cualquier análisis de correlación.

Gráficos de dispersión y correlación de variables: pairplot

pairplot() es una de las funciones más poderosas de Seaborn para analizar la relación entre varias variables numéricas al mismo tiempo. Genera una matriz de

gráficos de dispersión (scatterplots) para cada par de variables, y distribuciones en la diagonal.

```
sns.pairplot(df)
plt.show()
```

Parámetros importantes:

Parámetro	Función
hue	Distingue categorías con color
palette	Esquema de colores
kind	Tipo de gráfico (scatter, kde, etc.)
diag_kind	Tipo de gráfico en la diagonal (hist o kde)

Ejemplo con categoría:

```
sns.pairplot(df, hue='genero', palette='Set2')
```

Este tipo de gráfico es ideal para análisis exploratorio de datos (EDA), especialmente en modelos de machine learning, donde se busca visualizar la relación entre múltiples variables a la vez.

Gráficos de regresiones: regplot

regplot() permite representar gráficamente la relación entre dos variables continuas ajustando una línea de regresión lineal, además de visualizar los puntos de datos.

```
sns.regplot(x='edad', y='ingresos', data=df)
```

Características:

- Usa una regresión lineal por defecto.
- Admite ajuste de modelo con order (polinomial).
- Muestra el intervalo de confianza (por defecto al 95%).

Personalización:

Parámetro	Descripción
ci	Nivel de confianza (0 a 100)
line_kws	Personaliza la línea (color, grosor)
scatter_kws	Personaliza los puntos

```
sns.regplot(x='horas_estudio', y='nota_final', data=df, line_kws={'color':'red'})
```

Este gráfico es ideal cuando queremos ver si existe una tendencia creciente o decreciente entre dos variables, o si existe dispersión que justifique la no linealidad.

Gráficos de variables categóricas: barplot, countplot, boxplot, violinplot

Estas funciones están diseñadas para comparar categorías. A continuación las analizamos en profundidad.

barplot()

Representa el valor medio de una variable numérica para cada categoría.

```
sns.barplot(x='carrera', y='promedio', data=df)
```

- Calcula automáticamente la media y el intervalo de confianza.
- Útil para comparar promedios entre grupos.

countplot()

Cuenta la cantidad de observaciones por categoría.

```
sns.countplot(x='genero', data=df)
```

- No requiere variable numérica.
- Ideal para ver la distribución de frecuencias.

boxplot()

Muestra la mediana, cuartiles, valores atípicos y rango intercuartílico.

```
sns.boxplot(x='region', y='ingresos', data=df)
```

Elemento	Significado
Caja	Rango intercuartílico (Q1 a Q3)
Línea	Mediana
Bigotes	Rango sin outliers
Puntos	Outliers

Permite detectar valores extremos fácilmente.

violinplot()

Similar a boxplot, pero agrega la estimación de densidad (curva KDE), combinando información cuantitativa y distribución.

```
sns.violinplot(x='carrera', y='nota_final', data=df)
```

Diferencia principal

boxplot: muestra resumen estadístico

violinplot: añade distribución completa

Comparativa:

Función	Uso principal	Datos requeridos
barplot	Comparar promedios	Numérico + categórico
countplot	Contar observaciones	Solo categórico

boxplot	Visualizar dispersión y outliers	Numérico + categórico
violinplot	Ver distribución + resumen estadístico	Numérico + categórico

Gráficos de matrices: heatmap

heatmap() representa valores en forma matricial, usando color como variable cuantitativa. Es especialmente útil para visualizar matrices de correlación.

```
correlation = df.corr()
sns.heatmap(correlation, annot=True, cmap='coolwarm')
```

Parámetros clave:

Parámetro	Función
annot	Mostrar los valores sobre cada celda
cmap	Paleta de colores
linewidths	Líneas entre celdas
vmin, vmax	Escala de colores personalizada

Se utiliza ampliamente en análisis de variables para identificar relaciones significativas o multicolinealidad en modelos predictivos.

Grillas de gráficos: PairGrid y FacetGrid

Estas funciones permiten construir layouts complejos con múltiples gráficos de forma más flexible que pairplot() o displot().

PairGrid

Permite definir con mayor control qué función aplicar en cada parte de la matriz de gráficos.

```
g = sns.PairGrid(df, hue='genero')
g.map_diag(sns.histplot)
g.map_offdiag(sns.scatterplot)
g.add_legend()
```

FacetGrid

Divide el dataset en subgráficos según una variable categórica. Útil para comparación por grupos.

```
g = sns.FacetGrid(df, col='genero')
g.map(sns.histplot, 'edad')
```

También se puede usar con row, col, hue para crear estructuras más complejas.

Función	Diferencias clave
FacetGrid	Divide por categorías (columnas/filas)
PairGrid	Visualización cruzada de variables numéricas

Cierre ✍️

La visualización de datos es un componente esencial en todo proceso de análisis, desde la exploración inicial hasta la comunicación de resultados. Seaborn ofrece un conjunto robusto y accesible de herramientas para representar información de manera clara y efectiva.

A lo largo de este manual, aprendimos a identificar las características principales de Seaborn, importar y preparar nuestros datos, y construir diversos tipos de gráficos que nos permiten comprender y comunicar relaciones, distribuciones y patrones. Dominar estas técnicas proporciona una base sólida para el análisis estadístico visual en cualquier proyecto de ciencia de datos o ingeniería de datos.

Con la práctica constante, serás capaz de seleccionar la visualización adecuada para cada tipo de datos, personalizarla según tus necesidades, y enriquecer tus informes y dashboards con gráficos visualmente impactantes y técnicamente sólidos.

¡Nos vemos de nuevo, más adelante! 👋

Referencias📚

- Waskom, M. L. (2021). *seaborn: Statistical data visualization*. *Journal of Open Source Software*, 6(60), 3021. https://doi.org/10.21105/joss.03021
- Seaborn Documentation. (s.f.). *Seaborn documentation*. Recuperado en julio de 2025, de https://seaborn.pydata.org/
- Matplotlib Documentation. (s.f.). *Matplotlib documentation*. Recuperado en julio de 2025, de https://matplotlib.org/
- Pandas Documentation. (s.f.). *Pandas documentation*. Recuperado en julio de 2025, de https://pandas.pydata.org/

¡Muchas gracias!

Nos vemos en la próxima lección

alkemy